

Fase 10 — Docker e documentação final

Objetivo

Nesta fase, coloquei o projeto em containers para facilitar a execução e evitar a configuração manual do Python, MySQL e das dependências.

A ideia é conseguir subir o banco e o dashboard de forma mais simples usando Docker Compose.

O que foi feito

Dockerfile

O `Dockerfile` usa a imagem `python:3.11-slim` como base.

Nele são instaladas as dependências necessárias para o projeto, depois o arquivo `requirements.txt` é copiado e as bibliotecas Python são instaladas.

No final, o container inicia o dashboard feito com Streamlit na porta `8501`.

```
FROM python:3.11-slim
WORKDIR /app
...
CMD ["streamlit", "run", "src/dashboard/app.py", "..."]
```

docker-compose.yml

O Docker Compose ficou responsável por subir dois serviços:

Serviço	Imagem	Porta	Função
db	mysql:8.0	3306	Banco de dados MySQL
dashboard	build local	8501	Dashboard do projeto

O arquivo `sql/01_create_tables.sql` é montado no container do MySQL. Dessa forma, as tabelas são criadas automaticamente na primeira inicialização do banco.

Também adicionei um `healthcheck` no MySQL. O dashboard só é iniciado depois que o banco estiver disponível, evitando problemas de conexão durante a inicialização.

.env.example

Criei um arquivo `.env.example` com as variáveis utilizadas pelo projeto:

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=fiscalaudit
DB_USER=fiscalaudit
DB_PASSWORD=fiscalaudit
OPENAI_API_KEY=sua_chave_aqui
```

Para usar o projeto, basta copiar o arquivo para `.env` e preencher a chave da API quando necessário.

O arquivo `.env` não é versionado no Git, ficando apenas o `.env.example` no repositório.

Como executar

Com Docker

Essa é a forma mais simples de subir o ambiente.

```
# 1. Clonar o repositório
git clone https://github.com/ellen-xploit/fiscalaudit-ai.git
cd fiscalaudit-ai

# 2. Criar o arquivo de ambiente
cp .env.example .env

# 3. Subir os containers
docker compose up -d
```

Depois disso, o dashboard pode ser acessado em:

```
http://localhost:8501
```

Na primeira execução, o Docker precisa baixar as imagens e fazer o build da aplicação. Nas próximas execuções, o processo tende a ser mais rápido.

Para parar os containers:

```
docker compose down
```

Para parar os containers e remover também os volumes do banco:

```
docker compose down -v
```

Sem Docker

Também é possível executar o projeto diretamente no ambiente local.

```
# 1. Instalar as dependências
pip install -r requirements.txt

# 2. Criar o arquivo .env
cp .env.example .env

# 3. Gerar os dados fictícios
python src/gerador/gerar_dados_ficticios.py

# 4. Carregar os dados no banco
python src/etl/carregar_dados.py

# 5. Executar as regras de auditoria
python src/auditoria/regras.py

# 6. Rodar o modelo de ML
python src/ml/detector_anomalias.py

# 7. Gerar o relatório com IA
python src/ia/gerar_relatorio.py

# 8. Iniciar o dashboard
streamlit run src/dashboard/app.py
```

Estrutura final do projeto

```
fiscalaudit-ai/
├── data/
│   ├── raw/                # CSVs gerados pelo script de dados fictícios
│   └── processed/          # Resultados das regras de auditoria e ML
├── docs/                   # Documentação das fases do projeto
│   ├── fase_03_geracao_dados.md
│   ├── fase_04_etl.md
│   ├── fase_05_regras_auditoria.md
│   ├── fase_06_analise_exploratoria.md
│   ├── fase_07_machine_learning.md
│   ├── fase_08_ia_generativa.md
│   ├── fase_09_dashboard.md
│   └── fase_10_docker_documentacao.md
├── models/                 # Modelos treinados
├── notebooks/              # Notebooks da análise exploratória
├── sql/
│   └── 01_create_tables.sql # Criação das tabelas do banco
├── src/
│   ├── gerador/            # Geração dos dados fictícios
│   ├── etl/                # Pipeline de carga dos dados
│   ├── auditoria/          # Regras de auditoria e conciliação
│   ├── ml/                 # Detecção de anomalias
│   ├── ia/                 # Geração de relatório com LLM
│   └── dashboard/          # Dashboard em Streamlit
├── .env.example
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

Resumo das fases

Fase	Descrição	Entregável principal
1	Planejamento e modelagem do banco	README e modelagem relacional
2	Criação das tabelas no MySQL	<code>sql/01_create_tables.sql</code>
3	Geração de dados fictícios	<code>src/gerador/gerar_dados_ficticios.py</code>
4	Pipeline ETL	<code>src/etl/carregar_dados.py</code>
5	Regras de auditoria e conciliação	<code>src/auditoria/regras.py</code>
6	Análise exploratória	<code>notebooks/analise_exploratoria.ipynb</code>
7	Machine Learning	<code>src/ml/detect-or-anomalias.py</code>
8	IA Generativa	<code>src/ia/gerar_relatorio.py</code>
9	Dashboard	<code>src/dashboard/app.py</code>
10	Docker e documentação final	Dockerfile e docker-compose.yml